Python Complete Notes on

Comments, Variables, Keywords, Data Types, Operators, Input & Output Functions

---

1. COMMENTS IN PYTHON

Comments are used to explain code and make programs easier to understand.

Python ignores comments during execution.

Types of Comments

1. Single-Line Comment

Use #

```
# This is a comment
print("Hello")
```

---

2. Multi-Line Comment

Use triple quotes ''' ''' or """ """

```
"""
This is a
multi-line comment
"""

print("Python")
```

Why Comments Are Important

- Improves readability
- Helps debugging
- Makes teamwork easier
- Explains logic

Example

```
# Program to add two numbers
```

```
a = 10
b = 20

# Addition operation
c = a + b

print(c)
```

---

2. VARIABLES IN PYTHON

Variables are used to store data values.

Creating Variables

```
x = 10
name = "Priya"
price = 99.5
```

Python automatically decides the data type.

Rules for Naming Variables

Valid Rules

- Must start with letter or _
- Cannot start with number
- No spaces allowed
- Case-sensitive

Valid Examples

```
name = "Ram"
_age = 22
salary1 = 5000
```

Invalid Examples

```
1name = "Ram"     # Invalid
```

my name = "Ram" # Invalid

Multiple Variable Assignment

a, b, c = 10, 20, 30

print(a, b, c)

Assign Same Value

x = y = z = 100

print(x, y, z)

Swapping Variables

a = 10

b = 20

a, b = b, a

print(a)

print(b)

---

3. KEYWORDS IN PYTHON

Keywords are reserved words with special meanings.

We cannot use keywords as variable names.

Python Keywords List

False await else import pass

None break except in raise

True class finally is return

and continue for lambda try

as def from nonlocal while

assert del global not with

async elif if or yield

Checking Keywords

```
import keyword


print(keyword.kwlist)
```

Example

```
if 10 > 5:
   print("True")
```

Here if is a keyword.

---

4. DATA TYPES IN PYTHON

Data types specify the type of value stored.

Main Data Types

Data Type	Example
int	10
float	10.5
complex	2+3j
str	"Python"
list	[1,2,3]
tuple	(1,2,3)
set	{1,2,3}
dict	{"a":1}
bool	True
NoneType	None

4.1 Integer (int)

Stores whole numbers.

```
x = 100
print(type(x))
```

4.2 Float (float)

Stores decimal values.

```
price = 99.99
print(type(price))
```

4.3 Complex (complex)

```
z = 2 + 3j
print(type(z))
```

4.4 String (str)

Collection of characters.

```
name = "Python"
print(type(name))
```

String Operations

```
a = "Hello"
b = "World"

print(a + b)
print(a * 3)
```

4.5 List

Ordered and mutable collection.

```
numbers = [1, 2, 3]
print(numbers)
```

List Features

- Allows duplicates

- Changeable
- Ordered

4.6 Tuple

Ordered and immutable collection.

t = (1, 2, 3)

4.7 Set

Unordered collection without duplicates.

s = {1, 2, 3}

4.8 Dictionary

Stores data in key-value pairs.

```
student = {
   "name": "Ram",
   "age": 21
}


print(student)
```

4.9 Boolean

Stores True or False.

```
x = True
print(type(x))
```

4.10 NoneType

Represents no value.

```
x = None
print(type(x))
```

Type Conversion

Implicit Conversion

Python converts automatically.

```
a = 10

b = 2.5


print(a + b)
```

Explicit Conversion

Manual conversion.

```
x = "10"


print(int(x))
```

---

5. OPERATORS IN PYTHON

Operators are symbols used to perform operations.

Types of Operators

1. Arithmetic
2. Assignment
3. Comparison
4. Logical
5. Bitwise
6. Membership
7. Identity

5.1 Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction

Operator	Meaning
*	Multiplication
/	Division
%	Modulus
//	Floor Division
**	Exponent

Example

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a // b)
print(a ** b)
```

5.2 Assignment Operators

Operator	Example
=	x = 5
+=	x += 5
-=	x -= 5
*=	x *= 5
/=	x /= 5

Example

```
x = 10
```

```
x += 5

print(x)
```

5.3 Comparison Operators

Operator	Meaning
==	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater or Equal
<=	Less or Equal

Example

```
a = 10
b = 20

print(a == b)
print(a != b)
print(a < b)
```

5.4 Logical Operators

Operator	Meaning
and	Both True
or	Any One True
not	Reverse Result

Example

```
a = True
b = False

print(a and b)
print(a or b)
print(not a)
```

5.5 Bitwise Operators

Operator	Meaning
&	AND
|	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Example

```
a = 5
b = 3

print(a & b)
```

5.6 Membership Operators

Operator	Meaning
in	Present
not in	Not Present

Example

```
x = [1, 2, 3]
```

```
print(2 in x)

print(5 not in x)
```

5.7 Identity Operators

Operator	Meaning
is	Same Object
is not	Different Object

Example

```
a = [1,2]

b = a


print(a is b)
```

---

6. INPUT FUNCTIONS IN PYTHON

Input function is used to take data from the user.

---

input()

```
name = input("Enter your name: ")


print(name)
```

Important Point

input() always returns data as string.

Taking Integer Input

```
age = int(input("Enter age: "))


print(age)
```

Taking Float Input

```
salary = float(input("Enter salary: "))


print(salary)
```

Multiple Inputs

```
a, b = input("Enter two numbers: ").split()


print(a)
print(b)
```

Multiple Integer Inputs

```
a, b = map(int, input().split())


print(a + b)
```

---

7. OUTPUT FUNCTIONS IN PYTHON

Used to display data.

---

print()

```
print("Hello")
```

Printing Multiple Values

```
name = "Ram"
age = 21


print(name, age)
```

---

sep Parameter

```
print(1, 2, 3, sep="-")
```

Output:

1-2-3

---

end Parameter

print("Hello", end=" ")

print("World")

Output:

Hello World

Formatted Output

Using f-string

name = "Ram"

age = 21

print(f"My name is {name} and age is {age}")

Using format()

print("My name is {} and age is {}".format("Ram", 21))

Escape Characters

Escape Character	Meaning
\n	New Line
\t	Tab Space
\	Backslash
'	Single Quote
"	Double Quote

Example

print("Hello\nWorld")